



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 14 - Part 1

Templates





THIS WEEK'S AGENDA

- 01. Introduction to Templates
- 02. Template Functions
- 03. Templates with multiple arguments
- 04. Class Templates
- 05. Summary





INTRODUCTION TO TEMPLATES





INTRODUCTION TO TEMPLATES

- Consider the following function:

```
int absolute(int n) {  
    return (n < 0) ? -n : n;  
}
```

- Instead of stating the same function for various data types such as long:

```
long absolute(long n) {  
    return (n < 0) ? -n : n;  
}
```





INTRODUCTION TO TEMPLATES

- Or float:

```
float absolute(float n) {  
    return (n < 0) ? -n : n;  
}
```

- We can instead make the function generic for any data type that operates on the operations “<” and “-”.





INTRODUCTION TO TEMPLATES

- Such a function can be written as:

```
template <class T>
T abs(T n) {
    return (n < 0) ? -n : n;
}
```

- The statement `template<class T>` is known as a **function template** and the function is called a **template function**.
- The above function is *generic* in the sense that it is applicable for a generic type `T`.





INTRODUCTION TO TEMPLATES

- The key innovation in function templates is to represent the data type by a name that can stand in for any type.
- In the example we saw, this name was **T**.
- The template keyword that is used signals to the compiler that we are about to define a function template.
- The keyword class, within the angle brackets, might just as well be called a data type.
- The variable following the keyword class (**T** in our example) is called the template argument.





INTRODUCTION TO TEMPLATES

- Throughout the definition of the template we substitute the template argument, **T**.
- In the absolute() function template this name appears only twice, both in the first line (i.e. the function declarator).
- It appears as the **argument type** and the **return type**.
- In more complex functions it may appear numerous times throughout the function body as well.





TEMPLATE FUNCTIONS





HOW TO CALL A TEMPLATE FUNCTION?

- Let's look at an example:

```
template <class T>
T absolute(T n) {
    return (n < 0) ? -n : n;
}

int main() {
    int a = -10;
    double b = 2.1;
    std::cout << "Absolute of " << a << " is: " << absolute(a) << std::endl;
    std::cout << "Absolute of " << b << " is: " << absolute(b) << std::endl;
    return 0;
}
```





TEMPLATES WITH MULTIPLE ARGUMENTS

```
#include <iostream>

/* the function returns the index number
of the item, or -1 if not found. */
template <class atype>
int find(atype *array, atype value, int size) {
    for (int j = 0; j < size; j++) {
        if (array[j] == value) {
            return j;
        }
    }
    return -1;
}
```





TEMPLATES WITH MULTIPLE ARGUMENTS

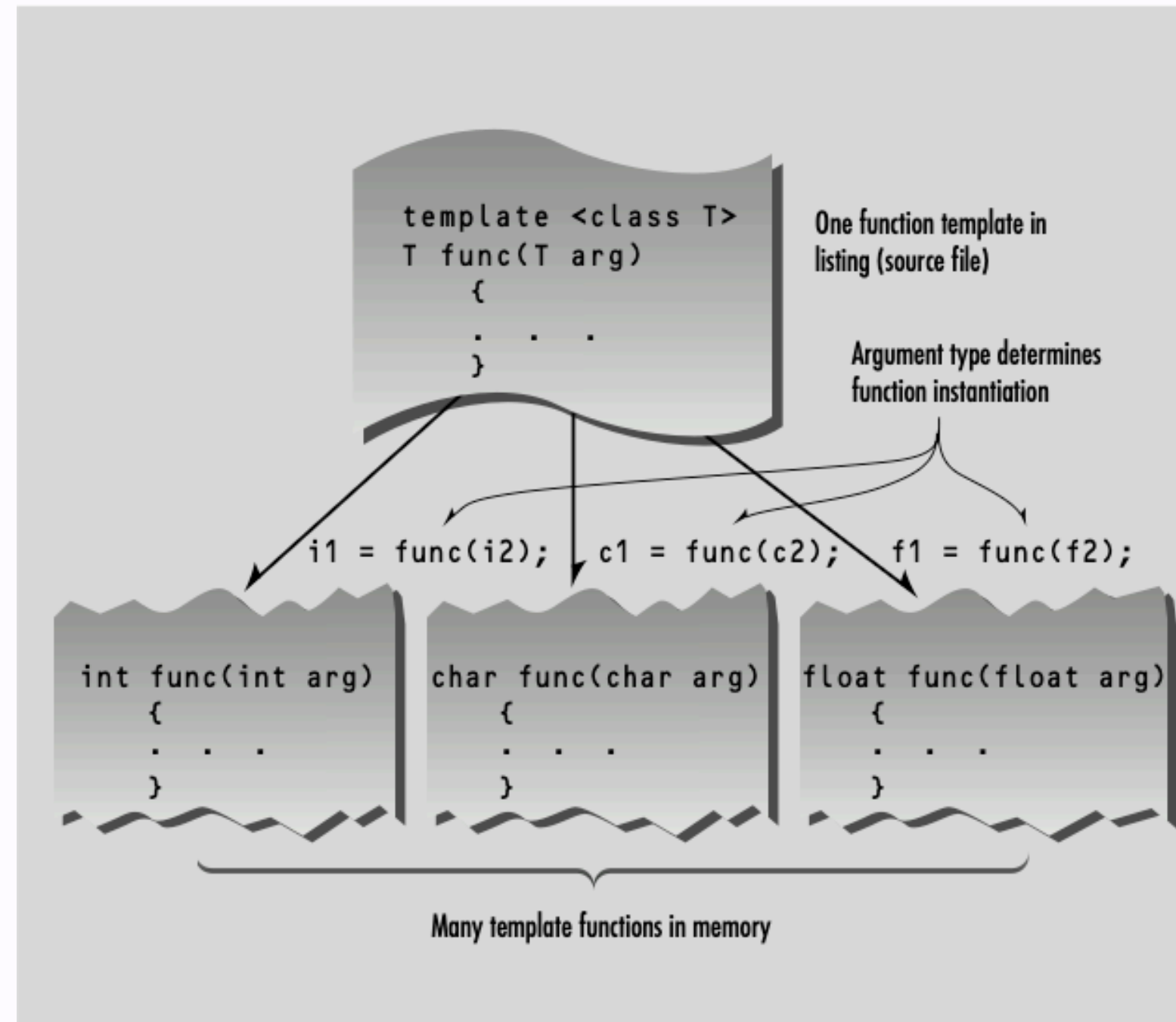
```
char chArr[] = {1, 3, 5, 9, 11, 13};
char ch = 5;
int intArr[] = {1, 3, 5, 9, 11, 13};
int in = 6;
long longArr[] = {1L, 3L, 5L, 9L, 11L, 13L};
long lo = 11L;
double doubleArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};
double db = 4.0;

int main() {
    std::cout << ch << " in chArr: index = " << find(chArr, ch, 6) << std::endl;
    std::cout << in << " in intArr: index = " << find(intArr, in, 6) << std::endl;
    std::cout << lo << " in longArr: index = " << find(longArr, lo, 6) << std::endl;
    std::cout << db << " in doubleArr: index = " << find(doubleArr, db, 6) << std::endl;
    return 0;
}
```





TEMPLATES WITH MULTIPLE ARGUMENTS





CLASS TEMPLATES





CLASS TEMPLATES

- The template concept can be extended to classes as well.
- Class templates are generally used for data storage (container) classes.
- Stacks and linked lists, which we have encountered in our previous classes are examples of data-storage classes.
- However, we have previously used them to store data of only a single basic data type. For example: a stack of integers, a linked list of doubles, etc.
- We can write a single class specification that would work for variables of all types, instead of a single basic data type.
- Class templates allow us to do this.
- Let's look at an example:





CLASS TEMPLATES

```
#include <iostream>

const int MAX = 100; // size of array
template <class Type>
class Stack {
    private:
        Type st[MAX]; // array of any type
        int top; // index of top of stack
    public:
        Stack() {top = -1;} // constructor
        void push(Type var) {
            st[++top] = var; // put number on stack
        }
        Type pop() {
            return st[top--]; // take number off stack
        }
};
```





CLASS TEMPLATES

```
int main() {  
    Stack<float> s1;  
  
    s1.push(11.2F);  
    s1.push(45.12F);  
    s1.push(21.90F);  
    std::cout << "1: " << s1.pop() << std::endl;  
    std::cout << "2: " << s1.pop() << std::endl;  
    std::cout << "3: " << s1.pop() << std::endl;  
  
    Stack<int> s2;  
  
    s2.push(11);  
    s2.push(21);  
    s2.push(90);  
    std::cout << "1: " << s2.pop() << std::endl;  
    std::cout << "2: " << s2.pop() << std::endl;  
    std::cout << "3: " << s2.pop() << std::endl;  
    return 0;  
}
```





SUMMARY





SUMMARY

- Templates allow us to write generic versions of functions and classes.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 14





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

